

TORCS AI Racing Competition

Student Setup Guide

From zero to racing in under 30 minutes

What you will achieve by following this guide:

- ✓ Install Podman (the container engine) on your machine
 - ✓ Download and run the competition container
- ✓ See TORCS running on screen with a car on track
 - ✓ Run the example AI agent and watch it drive
- ✓ Open VS Code with Granite AI assistant ready to help you code

Who is this guide for?

Absolute beginners. No prior knowledge of Docker, Podman, Linux, or TORCS is required. Every step is explained from scratch. If you can install an app on your computer, you can follow this guide.

0. What Is All This? (Read This First)

Before we start clicking and typing, here is a plain-English explanation of what you are about to set up and why each piece exists.

0.1 TORCS — the racing simulator

TORCS (The Open Racing Car Simulator) is an open-source car racing game that has been used for AI research for over a decade. Instead of a human pressing the keyboard, an AI program drives the car by reading sensor data (speed, track position, distances to walls) and sending commands (steer, accelerate, brake).

Your job in this competition is to write a Python program that drives a car around a track as fast and as cleanly as possible.

0.2 The container — your ready-made lab

Setting up TORCS from scratch on your own computer would take hours and involves compiling code from source. Instead, we have prepared a container — think of it as a pre-built virtual computer that already has everything installed inside it: TORCS, Python, VS Code, and an AI coding assistant.

You just need to run one command and the container starts up instantly, fully configured.

0.3 Podman — the engine that runs the container

Podman is the program you install on your own machine (Windows, Mac, or Linux) that is capable of running containers. It is similar to Docker if you have heard of that. You install Podman once, and then use it to download and run the competition container.

0.4 The desktop in your browser

Once the container is running, you access a full graphical desktop inside it through your web browser. No special software is needed beyond what you already have. You open a browser, go to a local address, and you see a Linux desktop where TORCS and VS Code are waiting for you.

Term	What it means in plain English
Container	A pre-built virtual computer with everything already installed

Podman	The engine on your machine that runs containers
TORCS	The racing simulator your AI will drive in
gym_torcs	Python code that lets your AI talk to TORCS
SCR / port 3001	The connection between your AI and TORCS (like a phone call)
VS Code	A code editor, like Word but for writing programs
Granite AI	An AI assistant built into VS Code that helps you write code

1. What You Need Before You Start

You will need the following before beginning. Read through this list and make sure everything is in order.

1.1 Hardware requirements

Requirement	Minimum
Computer	Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+)
RAM	8 GB minimum, 16 GB recommended
Free disk space	15 GB (the container image is large)
Internet connection	Required for the initial download only
CPU	Any modern 64-bit processor (Intel, AMD, or Apple Silicon)

Apple Silicon (M1/M2/M3) Mac?

This guide works on Apple Silicon Macs. There is a container built to run on ARM64 architecture as well as one for AMD64. No special steps are needed.

In case the setup of container environment is too much, you can also try to run TORCS in WINE environment (<https://www.winehq.org/>) – compatibility layer allowing to run Windows application on MacOs.

1.2 A web browser

You need any modern browser: Chrome, Firefox, Safari, or Edge. This is how you will see the desktop inside the container.

1.3 A terminal (command line)

You will need to type a few commands into a terminal window. Do not worry if you have never done this — every command is written out exactly as you need to type it.

- Windows: press the Windows key, type PowerShell, and press Enter
- Mac: press Cmd + Space, type Terminal, and press Enter
- Linux: press Ctrl + Alt + T

2. Installing Podman

Podman is the engine that runs your competition container. Follow the instructions for your operating system below. Only do the section that matches your computer.

2.1 Windows

1. Open your web browser and go to:

```
https://podman.io/docs/installation
```

2. Click the "Windows" section and download the latest Podman installer (.exe file).
3. Run the downloaded installer. Accept all default settings and click Next until the installation is complete.
4. When asked to enable WSL2 (Windows Subsystem for Linux), click Yes. WSL2 is required for Podman on Windows. Your computer may restart during this step.
5. After restarting, open PowerShell and type the following command to finish Podman setup:

```
podman machine init --disk-size 80 --cpus 4 --memory 8192
```

6. Then start the Podman machine:

```
podman machine start
```

7. Verify Podman is working by typing:

```
podman --version
```

You should see output like: podman version 5.x.x
(Use podman version 4.x and higher)

Windows Troubleshooting

If you see an error about virtualisation being disabled, you may need to enable it in your computer's BIOS/UEFI settings. Search online for 'enable virtualisation [your computer brand]' for instructions specific to your machine.

2.2 macOS

1. Open your web browser and go to:

```
https://podman.io/docs/installation
```

2. The easiest way on Mac is using Homebrew. If you do not have Homebrew, install it first by opening Terminal and pasting:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Once Homebrew is installed, install Podman by typing in Terminal:

```
brew install podman
```

4. Initialise the Podman virtual machine:

```
podman machine init --disk-size 80 --cpus 4 --memory 8192
```

5. Start the Podman machine:

```
podman machine start
```

6. Verify Podman is working:

```
podman --version
```

You should see output like: podman version 5.x.x

2.3 Linux (Ubuntu/Debian)

1. Open a terminal and run:

```
sudo apt-get update && sudo apt-get install -y podman
```

2. Verify Podman is working:

```
podman --version
```

On Linux, Podman runs natively without needing a virtual machine, so there is no 'machine init' step.

3. Preparing Your Workspace

Before running the container for the first time, you need to create a folder on your computer where your code will live. This folder is linked to the container so your work is always saved on your own machine — even when the container is stopped.

3.1 Create the RaceYourCode folder

Open your terminal and run the command for your operating system:

Mac or Linux:

```
mkdir ~/RaceYourCode
```

Windows (PowerShell):

```
mkdir C:\Users\YourName\RaceYourCode
```

(Replace YourName with your actual Windows username.)

3.2 Add the gym_torcs starter kit

1. Your competition organiser will provide a gym_torcs folder with the starter code.
2. Copy and unzip gym_torcs.zip into your RaceYourCode folder so it looks like this:

```
~/RaceYourCode/gym_torcs/torcs_jm_par.py      <- edit this  
~/RaceYourCode/gym_torcs/snakeoil3_gym.py  
~/RaceYourCode/gym_torcs/gym_torcs.py  
~/RaceYourCode/gym_torcs/practice.xml
```

Why this folder matters

Everything in your RaceYourCode folder on your computer is instantly visible inside the container at /home/student/workspace — and vice versa. Your work is never lost when the container stops.

3.3 Start the container (downloads automatically on first run)

You do not need to download the image separately first. The single command below pulls the image from Docker Hub if it is not already on your machine, then starts the container immediately — all in one step.

How long will the first run take?

Podman downloads approximately 10-15 GB on the first run. On a typical university or home connection this takes 15 to 45 minutes. Subsequent starts take only a few seconds.

Mac (arm64):

```
podman run -it --rm \  
  -p 5900:5900 -p 6080:6080 -p 3001:3001/udp \  
  -v ~/RaceYourCode:/home/student/workspace:Z \  
  --name torcs docker.io/johnsloe/torcs-competition:arm64
```

Linux (amd64):

```
podman run -it --rm \  
  -p 5900:5900 -p 6080:6080 -p 3001:3001/udp \  
  -v ~/RaceYourCode:/home/student/workspace:Z \  
  --name torcs docker.io/johnsloe/torcs-competition:amd64
```

Windows PowerShell:

```
podman run -it --rm \  
  -p 5900:5900 -p 6080:6080 -p 3001:3001/udp \  
  -v "C:\Users\YourName\RaceYourCode:/home/student/workspace:Z" \  
  --name torcs docker.io/johnsloe/torcs-competition:amd64
```

Download interrupted?

Simply run the same command again. Podman resumes from where it left off and will not re-download layers that already completed.

4. Starting the Container

If you completed section 3 and the container is already running, skip ahead to section 5. This section explains the start command in detail for reference.

4.1 The start command explained

Here is what each part of the command does — you do not need to memorise this, but it helps to understand:

Part of the command	What it does
podman run	Tells Podman to start a container
-it	Keeps the terminal connected so you can see output
--rm	Automatically removes the container when stopped (your files in RaceYourCode remain safe)
-p 6080:6080	Opens the browser desktop port
-p 5900:5900	Opens the VNC port (alternative desktop access)
-p 3001:3001	Opens the TORCS AI connection port
-v ~/RaceYourCode:/home/student/workspace:Z	Mounts your local folder into the container — live two-way sync
--name torcs	Gives the container a friendly name
docker.io/johnsloe/torcs-competition:tag	The image on Docker Hub — downloaded automatically if not already local

4.2 What you should see

After running the command, you will see several startup messages appearing in your terminal, something like:

```
[1/5] Starting Ollama (Granite 4.0 350M)...
[2/5] Starting virtual display...
[3/5] Starting XFCE desktop...
[4/5] Starting VNC server...
[5/5] Starting noVNC...
```

```
=====
Desktop ready!
```

```
Browser access : http://localhost:6080/vnc.html
```

When you see 'Desktop ready!' the container is running and waiting for you.

Important: leave this terminal window open!

The container runs inside this terminal. If you close the terminal window, the container will stop. Open a new terminal window for any other commands you need to run.

5. Accessing the Desktop

You access the full graphical desktop inside the container through your web browser. No extra software is needed.

5.1 Open the desktop in your browser

1. Open any web browser (Chrome, Firefox, Safari, or Edge).
2. In the address bar at the top, type exactly:

```
http://localhost:6080/vnc.html
```

3. Press Enter. You should see a connection screen. Click the blue Connect button.
4. A full Linux desktop (XFCE) will appear inside your browser window.

In case you locked your screen the username and password is “student”.

What is XFCE?

XFCE is a lightweight desktop environment for Linux. It looks different from Windows or macOS but works similarly — you have a taskbar, a file manager, and you can right-click on the desktop to open applications. You will quickly get used to it.

5.2 The desktop at a glance

Once connected, you will see:

- A desktop background with a taskbar at the bottom
- A file manager icon for browsing files
- Right-click on the desktop to access applications
- Your student workspace is at: `/home/student/workspace`

5.3 Opening a terminal inside the container

You will need a terminal inside the container to run your Python code. To open one:

1. Right-click anywhere on the desktop background.
2. Select Open Terminal from the menu.

A black terminal window will appear. This terminal is inside the container, running Linux. Commands you type here affect the container, not your own computer.

6. Running TORCS

Now we will launch TORCS and verify it runs correctly before connecting the AI agent to it.

6.1 Start TORCS from the terminal

1. In the terminal inside the container (see section 5.3), type:

```
torcs &
```

The & at the end runs TORCS in the background so you can continue typing other commands in the same terminal.

2. After a few seconds, the TORCS window will appear on the desktop.
3. You should see the TORCS main menu with options like Race, Options, and Quit.

6.2 Configure the race (first time only)

The first time you run TORCS, you need to configure the race to use the practice mode with the SCR server (which is the interface your AI connects to). Follow these steps carefully using the TORCS GUI:

1. In the TORCS main menu, click Race.
2. Click Practice.
3. Click Configure Race.
4. Select a track (any track will work, for example: Wheel-1).
5. For the robot/driver slot, select scr_server.
6. Click Accept and then back to the Practice menu.
7. Click New Race. TORCS will switch to a blue waiting screen with some text. This means it is waiting for your AI agent to connect.

The blue waiting screen is correct!

When TORCS shows a blue screen with text like 'SCR server waiting for connection', everything is working correctly. TORCS is paused and waiting for your Python AI agent to connect on port 3001. Do not close this screen — proceed to Section 7.

7. Running the Example AI Agent

With TORCS waiting for a connection, you can now run the example AI agent. This is a pre-written Python script that will drive the car automatically so you can see everything working end-to-end.

7.1 Navigate to the workspace

1. Open a second terminal tab or window inside the container (right-click the desktop, choose Open Terminal).

2. Navigate to the gym_torcs folder where the example code lives:

```
cd /home/student/workspace/gym_torcs
```

3. Check the files are there:

```
ls
```

You should see files including: torcs_jm_par.py, snakeoil3_gym.py, gym_torcs.py, and others.

7.2 Run the example agent

1. Run the example AI agent script:

```
python3 torcs_jm_par.py
```

2. Switch back to the TORCS window. Within a couple of seconds you should see the car start moving on the track.

3. In the terminal where you ran the Python script, you will see messages like:

```
Client connected on 3001.....
```

This means the AI agent has successfully connected to TORCS and is driving the car.

Congratulations!

If you can see the car moving in TORCS and the terminal shows 'Client connected', everything is working perfectly. You have successfully set up the competition environment.

7.3 Stop the agent

To stop the AI agent, click in the terminal window where it is running and press:

```
Ctrl + C
```

This will stop the Python script. You can then close TORCS by pressing Ctrl + C in the terminal where you ran the torcs & command, or by using the TORCS menu.

8. Using VS Code and the AI Coding Assistant

The container includes VS Code (a code editor) with the Continue.dev extension pre-installed. Continue.dev gives you an AI coding assistant powered by IBM Granite 4.0, running entirely inside the container with no internet required.

8.1 Opening VS Code

1. In the terminal inside the container, type:

```
code /home/student/workspace/gym_torcs
```

2. VS Code will open with the gym_torcs folder loaded. You will see the file list on the left side.
3. Click on torcs_jm_par.py to open the main competition starter file.

8.2 Using the AI assistant

The Continue.dev extension appears as a small icon in the left sidebar of VS Code (it looks like a play button or two arrows). Click it to open the AI chat panel.

1. Click the Continue icon in the left sidebar.
2. A chat panel opens on the right side. Type a question like:

```
What does the STEER_GAIN parameter do in this file?
```

3. Granite AI will read your code and explain it to you.

You can also highlight any part of the code, right-click, and choose Ask Continue to get an explanation of just that section.

The AI assistant works offline

Granite 4.0 350M runs entirely inside the container. It does not send your code to the internet. This means it works even without an internet connection during the competition.

You can also configure Granite served by <https://replicate.com/>. Use this code to get free credit: <https://replicate.com/invites/a8717bfe-2f3d-4a52-88ed-1356231cdf03>. Find suitable Granite code model, get API and configure it in Continue.dev extension. Credit should give you ~1M tokens free of charge.

9. Your First Modification

Now that everything is working, here is how to make your first change to the AI agent and see the effect on track.

9.1 Understanding the parameters

Open `torcs_jm_par.py` in VS Code. Scroll to the section labelled **USER CONFIGURABLE PARAMETERS** (around line 493). You will see:

```
TARGET_SPEED = 100      # Target speed in km/h
STEER_GAIN = 30         # Steering sensitivity
CENTERING_GAIN = 0.20   # How strongly car stays centred
BRAKE_THRESHOLD = 0.9   # When to brake in corners
GEAR_SPEEDS = [0, 20, 40, 80, 100, 180] # Gear shift points
ENABLE_TRACTION_CONTROL = True
```

9.2 Make a change

1. Try changing `TARGET_SPEED` from 100 to 150:

```
TARGET_SPEED = 150
```

2. Save the file with **Ctrl + S**.
3. Make sure TORCS is running and waiting (repeat section 6 if needed).
4. In the terminal, run the agent again:

```
python3 torcs_jm_par.py
```

5. Watch the car on track. It should now drive faster. Notice whether it stays on track or spins out — you may need to also adjust `STEER_GAIN` or `BRAKE_THRESHOLD` to compensate.

Tip: iterate quickly

The typical competition workflow is: change a parameter, save, run the agent, watch the car, stop the agent (**Ctrl+C**), change again. Each run only takes a few seconds to start. Do not be afraid to experiment.

9.3 What each parameter affects

Parameter	What to try
TARGET_SPEED	Increase for faster lap times, but the car may spin out in corners

STEER_GAIN	Increase if the car is drifting sideways, decrease if it oversteers
CENTERING_GAIN	Increase if the car wanders; decrease if it makes too sharp corrections
BRAKE_THRESHOLD	Decrease (e.g. 0.5) to brake earlier before corners
GEAR_SPEEDS	Adjust shift points for more aggressive or conservative gear changes
ENABLE_TRACTION_CONTROL	Set to False to see what happens without it (spoiler: wheelspin)

10. Saving Your Work

If you started the container using the command in section 3.3 (which includes the `-v` flag), your work is already being saved automatically. There is nothing extra to do.

Any file you save inside the container at `/home/student/workspace` is instantly and automatically saved to your RaceYourCode folder on your own computer. When you stop the container your code is safe.

How the two-way sync works

Your RaceYourCode folder and the container's `/home/student/workspace` folder are the same location — not a copy. Save on either side and it appears instantly on the other. This means you can also edit files directly on your computer using any editor you like.

10.1 Starting the container each session

Each time you want to work, open your terminal and run the same command from section 3.3. The image is already local so it starts in seconds:

Mac or Linux:

```
podman run -it --rm \
  -p 5900:5900 -p 6080:6080 -p 3001:3001 \
  -v ~/RaceYourCode:/home/student/workspace:Z \
  --name torcs docker.io/johnsloe/torcs-competition:latest
```

Windows PowerShell:

```
podman run -it --rm `
  -p 5900:5900 -p 6080:6080 -p 3001:3001 `
  -v "C:\Users\YourName\RaceYourCode:/home/student/workspace:Z" `
  --name torcs docker.io/johnsloe/torcs-competition:latest
```

10.2 If you forgot the `-v` flag

If you accidentally started the container without the `-v` flag, you can copy files out before stopping it. Open a new terminal on your own machine and run:

```
podman cp torcs:/home/student/workspace/gym_torcs/torcs_jm_par.py ./
```

This copies your modified file to the current folder on your machine.

11. Troubleshooting

Here are solutions to the most common problems students encounter.

Browser shows a blank page or cannot connect

Make sure the container is fully started. Wait for the 'Desktop ready!' message in your terminal before opening the browser. Also make sure you are typing the address correctly:

```
http://localhost:6080/vnc.html
```

Note: it is http not https. Some browsers may autocorrect to https — make sure it says http.

Python script says 'Connection refused' or keeps waiting

This means the Python agent cannot find TORCS. Check the following:

- TORCS must be running and showing the blue waiting screen before you run the Python script
- Make sure you started a New Race in Practice mode with scr_server as the driver
- Run both TORCS and the Python script inside the same container (in the browser desktop), not on your own machine

The car immediately goes off track and stops

This is normal behaviour with the default parameters on some tracks. The example agent is tuned for moderate-difficulty tracks. Try:

- Reducing TARGET_SPEED to 80 or 60
- Increasing CENTERING_GAIN to 0.4
- Choosing a simpler track in TORCS (a straight or gentle oval)

VS Code does not open

Make sure you are opening VS Code from the terminal inside the container (in the browser desktop), not from your own machine's terminal. Type:

```
code /home/student/workspace/gym_torcs
```

The AI assistant (Continue.dev) is not responding

The Granite model needs a moment to load when the container first starts. Wait 30 seconds after the desktop appears before using the AI assistant. If it still does not respond, open a terminal inside the container and check Ollama is running:

```
ollama list
```

You should see granite4:350m listed. If not, run:

```
ollama pull granite4:350m
```

Container is very slow

The container uses significant memory. Close any other heavy applications on your computer (especially browsers with many tabs). If you have less than 8 GB of RAM, performance may be limited.

12. Quick Reference Card

Keep this page handy. It summarises the most important commands.

Starting and stopping

Task	Command
Start container (Mac/Linux)	<code>podman run -it --rm -p 5900:5900 -p 6080:6080 -p 3001:3001 -v ~/RaceYourCode:/home/student/workspace:Z --name torcs docker.io/johnsloe/torcs-competition:latest</code>
Start container (Windows)	<code>podman run -it --rm -p 5900:5900 -p 6080:6080 -p 3001:3001 -v "C:\Users\YourName\RaceYourCode:/home/student/workspace:Z" --name torcs docker.io/johnsloe/torcs-competition:latest</code>
Access the desktop	Open browser: <code>http://localhost:6080/vnc.html</code>
Stop the container	Press <code>Ctrl + C</code> in the terminal where you started it

Inside the container

Task	Command (run in terminal inside container)
Start TORCS	<code>torcs &</code>
Go to workspace	<code>cd /home/student/workspace/gym_torcs</code>
Run the example agent	<code>python3 torcs_jm_par.py</code>
Stop the agent	<code>Ctrl + C</code>
Open VS Code	<code>code /home/student/workspace/gym_torcs</code>
Check Ollama models	<code>ollama list</code>

Key files

File	What it is
<code>/home/student/workspace/gym_torcs/torcs_jm_par.py</code>	Main starter file — edit this for the competition
<code>/home/student/workspace/gym_torcs/snakeoil3_gym.py</code>	Core UDP client — do not edit
<code>/home/student/workspace/gym_torcs/gym_torcs.py</code>	RL environment wrapper — advanced users
<code>/home/student/workspace/gym_torcs/practice.xml</code>	Race configuration

Good luck and happy racing!

If you get stuck, ask your competition organiser or use the Granite AI assistant in VS Code.